Project - LowLevel Design

On

Non Profit Reactive Triage Agent

Course Name:

Agentic-AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AAYUSH BANWADIKAR	EN22ME304002
2	DIVYANSHU MEHTA	EN22EL301020
3	ADITYA KUMAR SAHU	EN22ME304005
4	VINAY PATIDAR	EN22CS3011085
5	YATENDRA VERMA	EN22ME304118

Group Name: 02D10

Project Number: GAI-15

Industry Mentor Name: Mr. Ashween Krishnan

University Mentor Name: Prof. Maya Yadav Baniya

Academic Year: 2026

Table of Contents

1. Introduction.

2. Problem statement

3. Project Objectives

4. Proposed solution

5. Architecture Diagram

6. Key Technologies

7. Conclusion

1. Introduction

This Low-Level Design (LLD) presents the design and implementation of a Reactive Triage Agent developed to automate the initial handling of non-profit communications. The system leverages Large Language Models (LLMs) to intelligently process incoming messages by performing classification of urgency and intent, Named Entity Recognition (NER) for extracting key information, and generation of context-aware draft responses.

The solution is built using FastAPI for backend services, CrewAI for coordinating specialized agents, Groq for fast LLM inference, and spaCy for entity extraction, with a React-based user interface.The system follows a multi-agent, sequential processing architecture, ensuring modularity and clear separation of responsibilities. This document details the internal components, workflows, and interactions required to build an efficient and scalable AI-driven triage solution.

2. Project Objectives

The primary objective of this project is to design and develop a Reactive Triage Agent that automates the initial processing of non-profit communications using Large Language Models (LLMs). The system aims to improve efficiency, consistency, and response time in handling incoming messages.

Key Objectives:

- Automate message classification to identify urgency and intent for effective prioritization.
- Extract critical information using Named Entity Recognition (NER) such as names, dates, IDs, and organizations.
- Generate context-aware and empathetic draft responses using Large Language Models (LLMs).
- Implement a multi-agent architecture using CrewAI for structured workflow orchestration.
- Ensure scalable backend processing using FastAPI for efficient request handling.
- Leverage high-performance LLM inference using Groq for fast and reliable responses.
- Provide an interactive and responsive user interface using React.

3. Proposed Solution

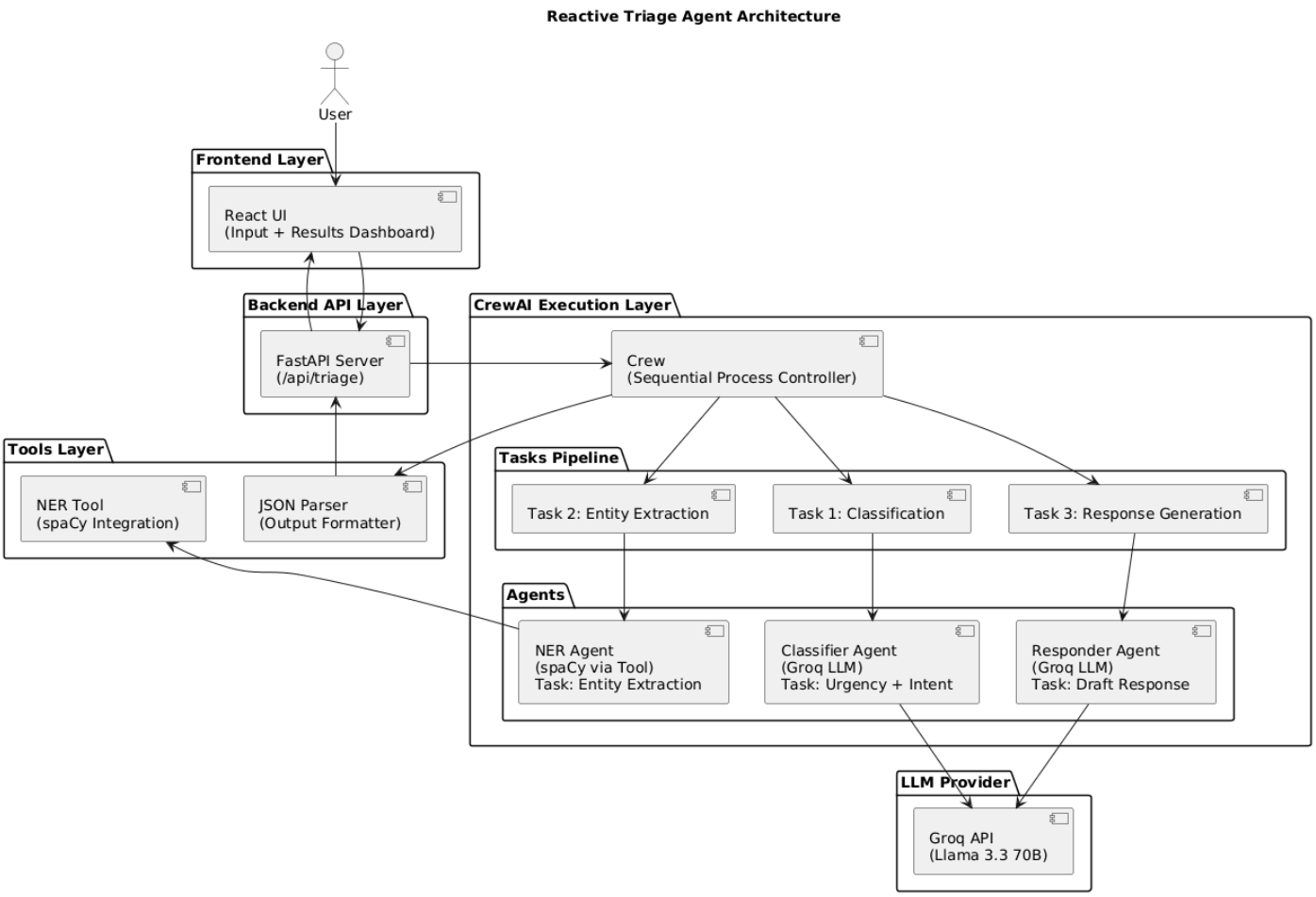
The proposed solution is a **Reactive Triage Agent** that automates the initial processing of non-profit communications using a multi-agent, LLM-driven architecture. The system accepts incoming messages through a user interface, processes them via a backend API, and applies a sequential pipeline of intelligent agents to generate structured outputs.

The solution utilizes CrewAI to coordinate multiple specialized agents, each responsible for a specific task. A classification agent leverages Groq (Llama 3.3 model) to determine the urgency and intent of the message. A data extraction agent uses spaCy to perform Named Entity Recognition (NER) and extract critical entities such as names, dates, and IDs. Finally, a response generation agent utilizes the LLM to produce a context-aware draft reply.

The backend is implemented using FastAPI, which exposes RESTful endpoints for processing requests and managing the agent workflow. The frontend is developed using React, providing an interactive interface for submitting inputs and visualizing outputs.

This modular and scalable design ensures efficient processing, clear separation of concerns, and easy extensibility, enabling non-profit organizations to significantly improve response time and communication management.

4. Architecture Diagram



The system is built using a multi-agent workflow, where each agent is responsible for a specific task in the triage pipeline.

4.1 Backend Components

4.1.1. NERTool (tools/ner_tool.py)

Class Name: NERTool
Extends: BaseTool (CrewAI)

Purpose:
Encapsulates the spaCy NLP pipeline as a reusable tool for extracting structured information from unstructured text.

Attributes:

- name: "Named Entity Recognition Tool"
- description: "Extracts named entities from email text"

Methods:

- **_run(text: str) → str**
  - Processes input using spaCy
  - Applies regex patterns to detect custom identifiers
  - Returns extracted entities in JSON format

4.1.2. Agent (CrewAI)

Class Name: Agent

Purpose:
Represents an autonomous AI entity with a defined role, goal, and reasoning capability.

Agents Implemented:

- **Classifier Agent**
  - Role: Email Triage Specialist
  - Function: Determines message intent and urgency

- **Extractor Agent**
  - Role: Data Extraction Specialist
  - Function: Extracts relevant entities from text
- **Responder Agent**
  - Role: Non-Profit Communications Officer
  - Function: Generates a contextual response

Attributes:

- role: Defines responsibility
- goal: Defines objective
- backstory: Context for reasoning
- llm: Language model used
- tools: Available tools
- verbose: Enables logging
- allow_delegation: Allows collaboration

4.1.3. Task (CrewAI)

Class Name: Task

Purpose:
Defines specific tasks assigned to agents.

Tasks Implemented:

- classify_task → Classifies urgency and intent
- extract_task → Extracts entities
- draft_task → Generates draft response

Attributes:

- description: Task definition
- expected_output: Output format
- agent: Assigned agent

4.1.4. Crew (CrewAI)

Class Name: Crew

Purpose:
Coordinates multiple agents and tasks in a structured workflow.

Attributes:

- agents: List of agents
- tasks: Ordered task sequence
- process: Sequential execution

- verbose: Logging

Methods:

- kickoff() → CrewOutput
  Initiates execution of the entire agent workflow

4.1.5. LLM Integration

Class Name: LLM

Purpose:
Provides abstraction for interaction with Large Language Models using LiteLLM.

Configuration:

- model: "groq/llama-3.3-70b-versatile"
- api_key: Secure API key
- temperature: Controls randomness

4.2. Frontend Components (React)

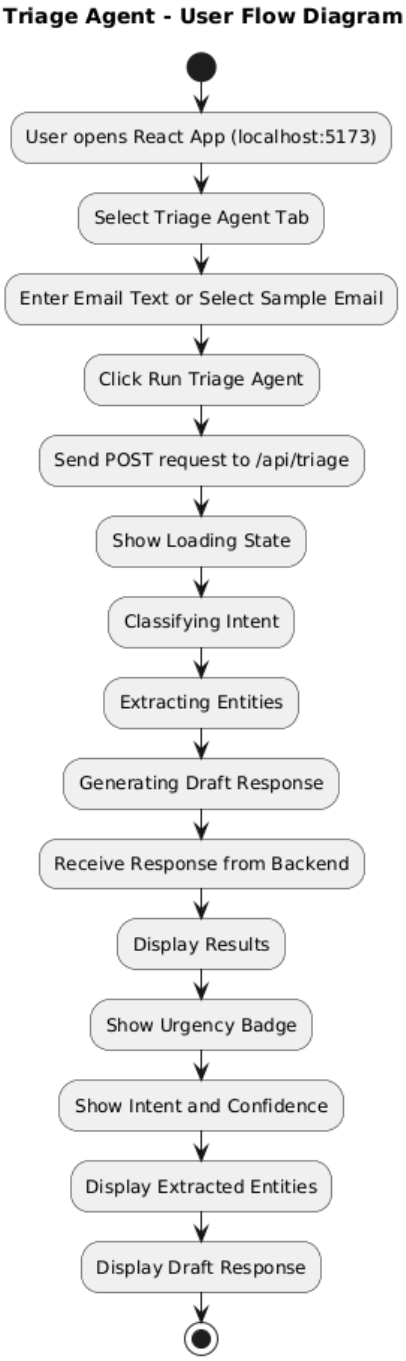
4.2.1. Core Components

- **App**
  - Manages navigation and global state
- **TriagePanel**
  - Handles user interaction and API calls
  - State: text, loading, step, result, error
  - Method: handleRun()

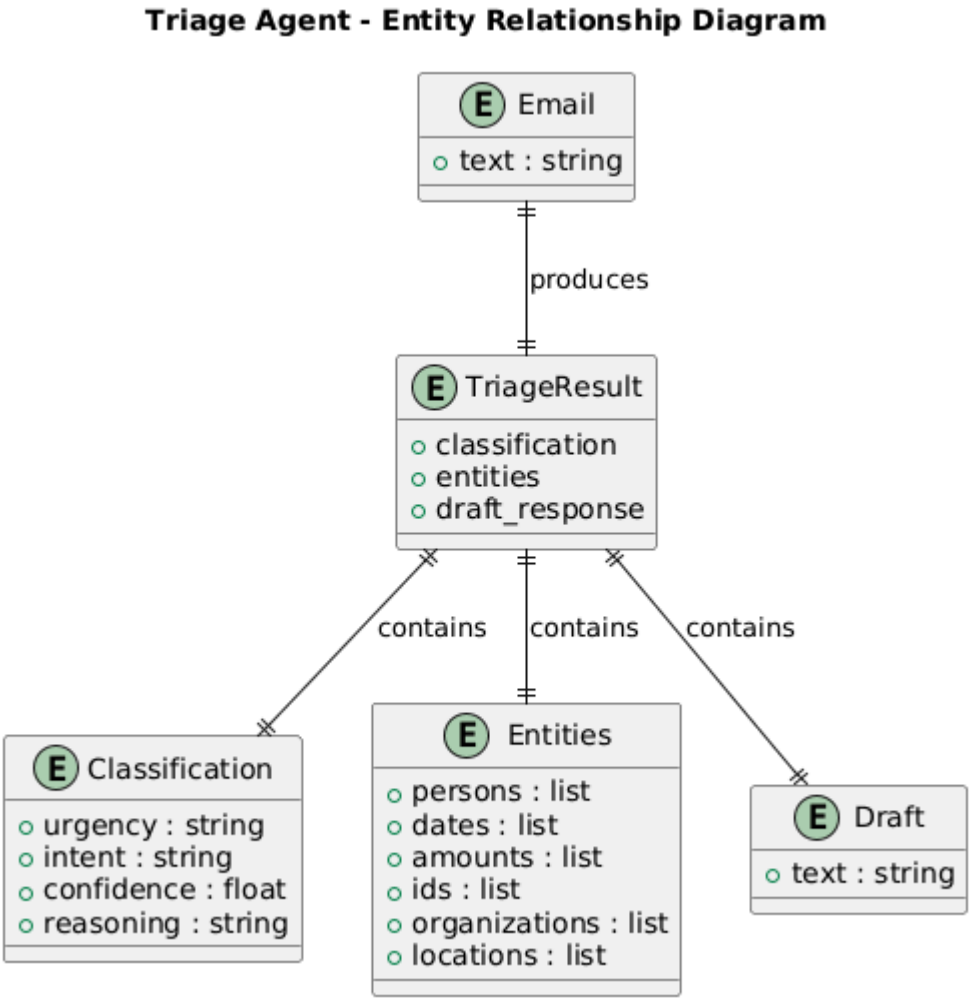
4.2.2. UI Components

- EmailInput → Input field for message
- SampleEmails → Predefined test inputs
- ClassificationCard → Displays intent and urgency
- EntityCard → Displays extracted entities
- DraftResponse → Shows generated reply

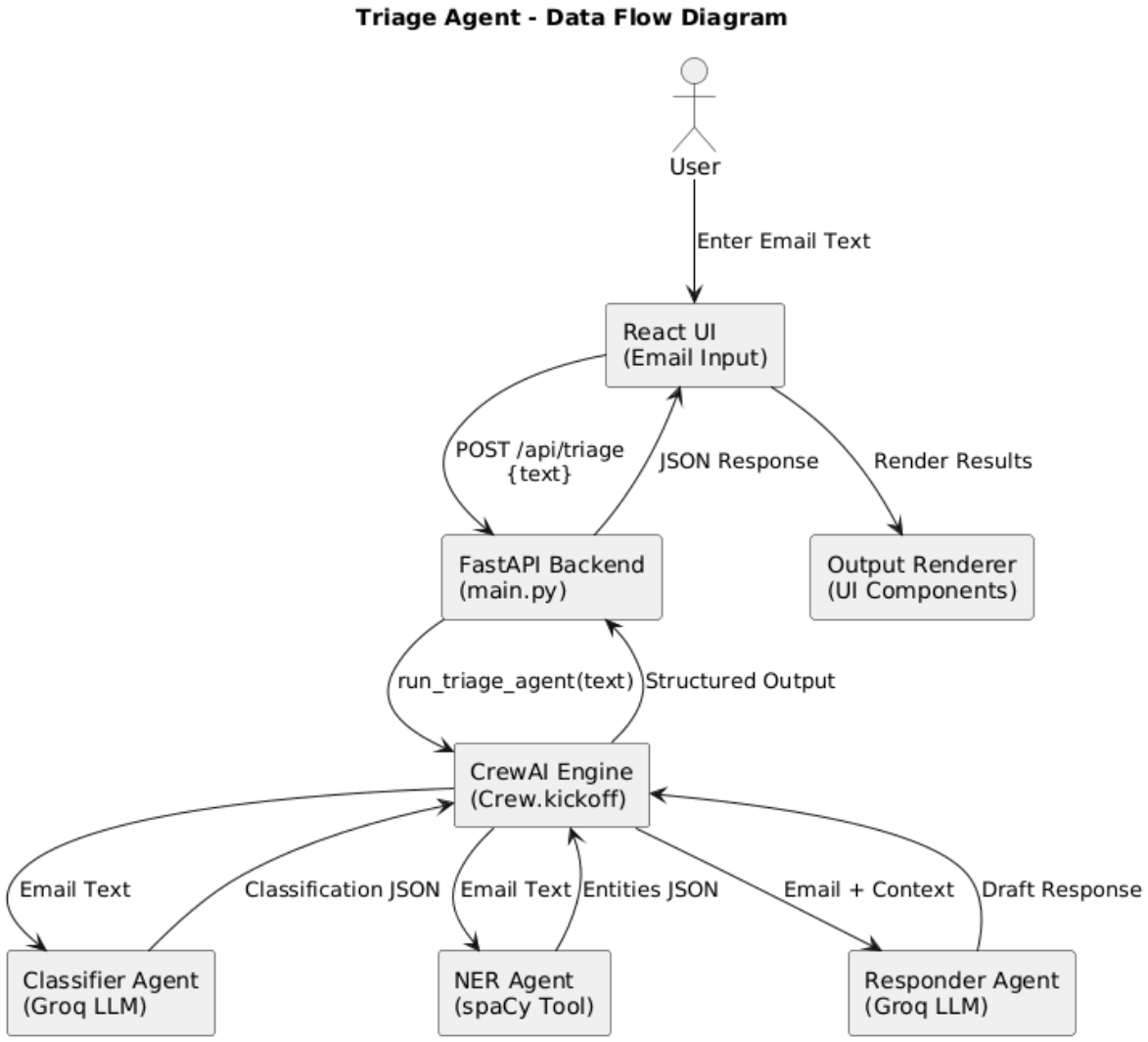
5. User Flow Diagram



6. Entity Relationship Diagram



7. Data Flow Diagram



8. Key Technologies

- **FastAPI:** Used as the backend framework to develop RESTful APIs. It handles incoming requests from the frontend, manages routing (e.g., /api/triage), performs input validation, and returns structured responses.
- **CrewAI:** Acts as the core orchestration layer, enabling a multi-agent architecture. It manages the execution of specialized agents (classification, entity extraction, and response generation) in a sequential workflow.

- **Groq (Llama 3.3 Model):** Provides high-performance Large Language Model (LLM) inference. It is used for message classification (urgency and intent) and context-aware response generation, ensuring fast and accurate outputs.
- **spaCy:** Used for Natural Language Processing tasks, specifically Named Entity Recognition (NER). It extracts structured information such as names, dates, IDs, organizations, and other relevant entities from unstructured text.
- **React:** Serves as the frontend framework to build an interactive user interface. It allows users to input messages and view results including classification, extracted entities, and generated responses.
- **LangChain:** Facilitates seamless integration with the Groq LLM, handling model configuration and enabling smooth interaction between backend logic and LLM services.

9. Conclusion

This Low-Level Design (LLD) defines the architecture and internal workflow of the proposed Reactive Triage Agent for non-profit communications. It outlines a structured multi-agent approach for handling classification, entity extraction, and response generation using modern AI technologies.The design ensures modularity, scalability, and clear separation of concerns, providing a strong foundation for implementation and future enhancements.